

RAII — Խորը ուղեցույց

Resource Acquisition Is Initialization

Այս ուղեցույցը մանրամասն բացատրում է RAII-ի՝ ինչ է, ինչպես է աշխատում, ինչու է C++-ի ամենակարգիվ իդիոմ-ներից, իրականում (memory, file, lock, mutex), եւ interview հարցերով.

1. Ի՞նչ է RAII-ը

RAII = **Resource Acquisition Is Initialization**. C++-ի հիմնական իդիոմ, որտեղ resource-ի (memory, file, lock, socket) **կապում է object-ի կյանքի հետ**.

```
Resource ստացում է -> constructor-ում (object ստեղծվելիս)
Resource ազատում է -> destructor-ում (object ոչնչվելիս)
```

Object-ի scope-ից դուրս գալիս -> destructor ավտոմատ կանչվում -> resource ավտոմատ ազատվում.
Չկա մոռացում, չկա leak.

2. Խնդիրը ահա RAII

```
void f() {
    int* p = new int[100];
    FILE* file = fopen("data.txt", "r");

    if (error) return;    // LEAK -- delete[] u fclose չեն կանչվում
    mayThrow();           // exception -> նույն xndir

    delete[] p;
    fclose(file);
}
```

Ամեն exit path-ում (return, exception, break) թե՛ դեղձով ազատել -> հեշտ է մոռանալ -> leak / resource leak.

3. RAII-ov lucumy

```
class IntArray {
    int* data;
public:
    IntArray(int size) { data = new int[size]; }    // vernum ctor-um
    ~IntArray() { delete[] data; }                  // azatum dtor-um
};

void f() {
    IntArray arr(100);
    if (error) return;    // ochmi leak -- dtor avtomat kanchum
    mayThrow();           // exception -> stack unwinding -> dtor kanchum
}                          // scope verj -> dtor -> delete[]
```

Kariny vory ashxatum e nuynisk exception-i depqum. **stack unwinding** — exception-i zham C++-y avtomat kanchum e bolor local object-neri destructor-nery.

4. RAII real orinakner

4.1 Memory (smart pointer)

```
{
    auto p = std::make_unique<int>(5);
}    // avtomat delete
```

4.2 File

```
class File {
    FILE* f;
public:
    File(const char* name) { f = fopen(name, "r"); }
    ~File() { if (f) fclose(f); }    // avtomat close
};

void read() {
    File file("data.txt");
    // ... ogtagorcum
}    // avtomat fclose, nuynisk exception-i depqum
```

4.3 Mutex / Lock (amenakarevor)

```
std::mutex mtx;

void safe() {
    std::lock_guard<std::mutex> lock(mtx);    // ctor -> lock
    // critical section
}    // dtor -> unlock (avtomat, nuynisk exception-i depqum)
```

`std::lock_guard` -y RAII e. lock ctor-um, unlock dtor-um. Chka "mo'racu unlock" -> chka deadlock.

5. RAII և Rule of Zero

Lavaguyn RAII-n ayú e, erb dzerqov ochmi ban chi grum. STL tiper1 (vector, string, unique_ptr) ir1 RAII en.

```
class Widget {
    std::vector<int> data;    // RAII inqny
    std::string name;    // RAII inqny
    std::unique_ptr<int> ptr;    // RAII inqny
};    // ochmi destructor petq che -- Rule of Zero
```

Rule of Zero — ete ogtagorcum es RAII tiper, ochmi dzerqov destructor/copy/move petq che; compiler-i default-nery chisht en.

6. Inchu RAII amenakarevorn e C++-um

- Exception-safe (stack unwinding -> avtomat cleanup)
 - Chka moracum (deterministic destruction)
 - Chka leak (memory, file, lock, socket)
 - Chka manual cleanup amen exit path-um
 - Deterministic (GC-ic tarber` konkret pahin azatum)
-

7. Interview harcer & patasxanner

RAII inch e?

Resource Acquisition Is Initialization. resource-y kapvum e object-i kyanqin. vercvum ctor-um, azatvum dtor-um; scope verjum avtomat cleanup.

Inchu e petq?

Exception safety + chka leak/moracum. Amen exit path-um (return/throw) resource-y avtomat azatvum.

Inchpes e ashxatum exception-i depqum?

Stack unwinding -> C++-y avtomat kanchum e local object-neri destructor-nery -> cleanup.

RAII orinakner?

unique_ptr/shared_ptr (memory), lock_guard (mutex), fstream (file), vector/string.

RAII vs manual cleanup?

Manual amen path-um dzeŕqov, hesht moŕanal, exception-i depqum leak. RAII avtomat, exception-safe, deterministic.

Rule of Zero-i het kapy?

RAII tiper ogtagorcelis ochmi dzeŕqov dtor/copy/move petq che -> Rule of Zero.

Cheat Sheet

RAII = Resource Acquisition Is Initialization

resource vercvum -> constructor | azatvum -> destructor

scope verj / exception -> avtomat cleanup (stack unwinding)

ORINAKNER:

unique_ptr/shared_ptr -> memory

lock_guard/unique_lock -> mutex

fstream -> file

vector/string -> memory

AŔAVELUTYUN:

exception-safe | chka leak | chka moŕacum | deterministic

RULE OF ZERO: RAII tiper -> ochmi dzeŕqov dtor/copy/move